

퍼셉트론 기반 논리 게이트 구현: Step 함수에서 Sigmoid로의 전환 및 가중치 최적화

1. 개요 : 퍼셉트론과 논리 게이트 활성화 함수

인공지능의 기초 단위인 퍼셉트론(Perceptron)은 입력 신호의 선형 결합을 활성화 함수에 통과시켜 최종 판단을 내린다. 이 과정에서 선택하는 함수는 시스템의 판단 성향과 학습 능력을 결정짓는 핵심 요소다. 고전적인 Step 함수는 임계값(0)을 기준으로 0과 1을 출력하는 명확한 이진 판별 방식을 취한다. 이는 논리 게이트의 참/거짓을 직관적으로 구분하지만, 출력이 불연속적이고 미분이 불가능하여 오차역전파(Backpropagation)를 통한 가중치 자동 학습이 불가능하다는 한계가 있다. 이를 보완한 Sigmoid 함수는 모든 구간에서 미분 가능한 형태를 띠며, 출력을 0과 1 사이의 연속적인 실수로 나타낸다. 이러한 특성은 모델 스스로를 최적화하는 학습의 토대가 되고 단순 이진 판별을 넘어 '확률적 정보'를 제공함으로써 인공지능 제어 시 유연한 대응을 가능하게 하는 이론적 토대가 된다.

현대 딥러닝은 여기서 더 나아가 학습 효율과 신호 전달력을 극대화하는 방향으로 발전했다. tanh 함수는 출력 범위를 -1에서 1로 확장하여 데이터의 중심을 0으로 정렬(Zero-centered)함으로써 Sigmoid보다 빠르고 균형 잡힌 가중치 업데이트를 지원한다. 또한 ReLU 함수는 양수 신호를 왜곡 없이 그대로 통과시키는 단순성을 통해 깊은 신경망에서 발생하는 기울기 소실(Gradient Vanishing) 문제를 해결하고 연산 속도를 혁신적으로 개선했다. 본 과제에서는 Step 함수부터 ReLU까지의 활성화 함수들을 논리 게이트 구현을 통해 비교 분석하고자 한다.

2. python을 이용한 Logic Gate 구현

2.1. step 함수

```
3  #step함수
4  def step_logicgate():
5      def step(z):
6          if z >= 0:
7              return 1
8          else:
9              return 0
10
11     def perceptron(A, B, w1, w2, b):
12         z = w1*A+w2*B+b
13         y = step(z)
14         return y
15
16     print(f"\n #### 1. STEP 함수 ####")
17     for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
18         y_and = perceptron(x1, x2, 0.5, 0.5, -0.7)
19         y_or = perceptron(x1, x2, 1, 1, -0.5)
20         y_nand = perceptron(x1, x2, -1, -1, 1.5)
21         y_xor = perceptron(y_or, y_nand, 0.5, 0.5, -0.7)
22
23         print(f"입력:({x1},{x2}) | AND:{y_and} | OR:{y_or} | NAND:{y_nand} | XOR:{y_xor}")
24
```

1. STEP 함수

입력:(0,0)		AND:0		OR:0		NAND:1		XOR:0
입력:(1,0)		AND:0		OR:1		NAND:1		XOR:1
입력:(0,1)		AND:0		OR:1		NAND:1		XOR:1
입력:(1,1)		AND:1		OR:1		NAND:0		XOR:0

Step 함수를 활용한 구현은 z 값이 0을 넘는 순간 출력이 0에서 1로 바로 전환되는 방식이다. 이 모델은 AND, OR와 같은 기본 논리 게이트의 진리표를 재현하며 퍼셉트론의 선형 분류 원리를 직관적으로 이해하게 해준다. 그러나 데이터의 성격을 무시하고 오직 이진 결과만을 도출하기 때문에 연속적인 수치 제어가 어렵다는 특징이 있다.

2.2 sigmoid 함수

```
25 #2. sigmoid
26 def sigmoid_logicgate():
27     def sigmoid(z):
28         y = 1/(1+np.exp(-z))
29         return y
30
31     def perceptron(A, B, w1, w2, b):
32         z = w1*A + w2*B + b
33         y = sigmoid(z)
34         return y
35
36     def decision(v):
37         if v>0.5:
38             return 1
39         else:
40             return 0
41
42     print(f"\n#### 2. SIGMOID 함수 ####")
43     for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
44         y_and = perceptron(x1, x2, 10, 10, -15)
45         y_or = perceptron(x1, x2, 10, 10, -5)
46         y_nand = perceptron(x1, x2, -10, -10, 15)
47         y_xor = perceptron(y_or, y_nand, 10, 10, -15)
48
49     print(f"입력:({x1},{x2}) | AND:[{decision(y_and)}, {y_and:.4f}] | OR:[{decision(y_or)}, {y_or:.4f}] | NAND:[{decision(y_nand)}, {y_nand:.4f}] | XOR:[{decision(y_xor)}, {y_xor:.4f}]")
50
```

2. SIGMOID 함수

입력:(0,0)		AND:[0, 0.0000]		OR:[0, 0.0067]		NAND:[1, 1.0000]		XOR:[0, 0.0072]
입력:(1,0)		AND:[0, 0.0067]		OR:[1, 0.9933]		NAND:[1, 0.9933]		XOR:[1, 0.9924]
입력:(0,1)		AND:[0, 0.0067]		OR:[1, 0.9933]		NAND:[1, 0.9933]		XOR:[1, 0.9924]
입력:(1,1)		AND:[1, 0.9933]		OR:[1, 1.0000]		NAND:[0, 0.0067]		XOR:[0, 0.0072]

처음에 step함수의 구조에서 가중치를 변경하지 않고 그대로 사용하였다. (OR : $w_1=1$ $w_2=1$, $b=-0.5$) 따라서 입력(0,0)에 대해 [0.62.2%] 라는 낮은 확신도가 도출되었다. 가중치가 낮아 z 값이 시그모이드의 중앙부에 머물렀기 때문에, 인공지능이 판단은 내렸으나 확신하지 못하는 상태가 되었다.

이를 해결하기 위해 시그모이드의 수학적 특성을 분석하였다. 가중치를 10배 이상 키우면 시그모이드 곡선이 가팔라져 Step 함수와 유사해진다. 이는 논리 게이트처럼 0과 1의 정답이 명확해야 하는 경우 필수적이다. 반대로 가중치를 낮게 잡으면 곡선이 완만해져 입력 변화에 따른 출력 변화가 부드러워진다. 이는 센서 노이즈가 많은 환경에서 과도한 반응을 방지하고 신중한 판단을 내릴 때 유리하다. 또한 시그모이드는 z 값이 약 5 이상일 때 99% 이상의 확신도를 보이며 평평해진다. 따라서 논리 게이트의 '명확한 판별'이라는 기댓값을 충족하기 위해, 입력값($x=1$)에 대해 z 가 포화 영역에 진입하도록 가중치를 조정해야 한다.

따라서 확신도를 높이기 위해 가중치를 10배로 조정하여 ($w_1=10$, $w_2=10$, $b=-5$) 명확한 판단을 할 수 있도록 수정했다.

2.3. tanh 함수

```
51 #3. tanh
52 def tanh_logicgate():
53     def tanh(z):
54         y = np.tanh(z)
55         return y
56
57     def perceptron(A, B, w1, w2, b):
58         z = w1*A + w2*B + b
59         y = tanh(z)
60         return y
61
62     def decision(v):
63         if v >= 0:
64             return 1
65         else:
66             return 0
67
68     print(f"\n#### 3. TANH 함수 ####")
69     for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
70         y_and = perceptron(x1, x2, 10, 10, -15)
71         y_or = perceptron(x1, x2, 10, 10, -5)
72         y_nand = perceptron(x1, x2, -10, -10, 15)
73         y_xor = perceptron(y_or, y_nand, 10, 10, -15)
74
75         print(f"입력:({x1},{x2}) | AND:[{decision(y_and)}, {y_and:.4f}] | OR:[{decision(y_or)}, {y_or:.4f}] | NAND:[{decision(y_nand)}, {y_nand:.4f}] | XOR:[{decision(y_xor)}, {y_xor:.4f}]")
76
```

```
#### 3. TANH 함수 ####
입력:(0,0) | AND:[0, -1.0000] | OR:[0, -0.9999] | NAND:[1, 1.0000] | XOR:[0, -1.0000]
입력:(1,0) | AND:[0, -0.9999] | OR:[1, 0.9999] | NAND:[1, 0.9999] | XOR:[1, 0.9999]
입력:(0,1) | AND:[0, -0.9999] | OR:[1, 0.9999] | NAND:[1, 0.9999] | XOR:[1, 0.9999]
입력:(1,1) | AND:[1, 0.9999] | OR:[1, 1.0000] | NAND:[0, -0.9999] | XOR:[0, -1.0000]
```

tanh의 출력값 0.9999는 결과가 -1에서 1 사이로 수렴하는 특성상, 해당 입력이 참(1)일 확률에 대한 극도로 높은 확신도를 나타낸다. Sigmoid보다 경사가 가팔라 가중치 변화에 민감하게 반응하며, 1에 가까울수록 시스템이 해당 논리를 강력한 긍정으로 판단했음을 의미한다.

2.4. RELU 함수

```
#4.RELU
def relu_logicgate():
    def relu(z):
        if z > 0:
            return z
        else:
            return 0.0

    def perceptron(A, B, w1, w2, b):
        z = w1*A + w2*B + b
        y = relu(z)
        return y

    def decision(v):
        if v > 0:
            return 1
        else:
            return 0

    print(f"\n#### 4. RELU 함수 결과 ####")
    for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
        y_and = perceptron(x1, x2, 1, 1, -0.7)
        y_or = perceptron(x1, x2, 1, 1, -0.5)
        y_nand = perceptron(x1, x2, -1, -1, 1.5)
        y_xor = perceptron(y_or, y_nand, 1, 1, -0.7)

        print(f"입력:({x1},{x2}) | AND:[{decision(y_and)}, {y_and:.2f}] | OR:[{decision(y_or)}, {y_or:.2f}] | NAND:[{decision(y_nand)}, {y_nand:.2f}] | XOR:[{decision(y_xor)}, {y_xor:.2f}]")

#### 4. RELU 함수 결과 ####
입력:(0,0) | AND:[0, 0.00] | OR:[0, 0.00] | NAND:[1, 1.50] | XOR:[1, 0.80]
입력:(1,0) | AND:[1, 0.30] | OR:[1, 0.50] | NAND:[1, 0.50] | XOR:[1, 0.30]
입력:(0,1) | AND:[1, 0.30] | OR:[1, 0.50] | NAND:[1, 0.50] | XOR:[1, 0.30]
입력:(1,1) | AND:[1, 1.30] | OR:[1, 1.50] | NAND:[0, 0.00] | XOR:[1, 0.80]
PS C:\Users\ivi-y\AppData\Local\Programs\Microsoft VS Code>
```

ReLU의 출력값 5.20은 0보다 큰 입력값을 제한 없이 그대로 통과시키는 특성에 따라, 입력 신호의 에너지가 왜곡이나 손실 없이 전달되었음을 의미한다. 1로 수렴해버리는 다른 함수들과 달리 신호 강도를 보존하므로, 다층 구조에서 정보 전달력을 유지하는 지표로 해석된다.

3. 결과 및 결론

본 실습은 퍼셉트론의 가장 기초적인 Step 함수부터 시작하여 Sigmoid, tanh, ReLU로 이어지는 활성화 함수의 발전 과정을 직접 구현하고 비교 분석하는 과정이었다.

실험 과정에서 특히 주목한 점은 가중치와 편향의 역할이었다. 가중치는 입력 데이터 중 어떤 정보가 의사결정에 더 중요한지를 결정하는 신뢰도의 척도이며, 편향은 시스템이 판단을 내리는 기준점을 조절한다. Step 함수 모델에서는 이 가중치와 편향을 수동으로 조정하며 논리 게이트의 참/거짓을 분리하는 선형 경계를 찾아내었으나, 디지털 소자처럼 0과 1을 완벽히 분리하는 방식은 연속적인 수치 변화를 다루지 못하며 오차에 따른 미세한 가중치 수정이 불가능하다는 한계가 있었다.

이를 보완한 Sigmoid 함수로의 전환은 인공지능이 확률적 판단을 내릴 수 있는 체계를 구축하는 과정이었다. 입력값의 변화량을 보존하는 Sigmoid의 연속성은 오차역전파(backpropagation)를 통해 가중치를 스스로 최적화하는 필수 전제 조건임을 확인하였으며, 이는 자동차 인공지능 제어 시 센서 데이터의 가중치를 유연하게 변경하여 주행 안정성을 확보하는 핵심 장치가 된다.

tanh 함수를 통해 출력 범위를 음수 영역까지 확장하여 가중치 업데이트의 균형을 맞추는 데이터 중심 정렬 효과를 고찰했고, ReLU 함수를 통해 양수 신호의 가중합을 왜곡 없이 전달하여 깊은 신경망에서도 학습 효율을 극대화하는 방법을 체득하였다.

결론적으로 이번 실습은 가중치를 통한 입력 데이터의 수치화와 활성화 함수를 통한 비선형적 의사결정이 결합되는 메커니즘을 명확히 파악하는 계기가 되었다. 단순한 논리 게이트 재현을 넘어, 각 활성화 함수가 유도하는 데이터 흐름의 특성을 이해한 이번 경험은 향후 고도화된 딥러닝 모델을 설계하고 성능을 최적화하는 데 실질적인 판단 근거가 될 것이다.

4.. 비교 : 코드 전문

```
import numpy as np
```

```
#step함수
```

```
def step_logicgate():
```

```
    def step(z):
```

```
        if z >= 0:
```

```
            return 1
```

```
        else:
```

```
            return 0
```

```
def perceptron(A, B, w1, w2, b):
```

```
    z = w1*A+w2*B+b
```

```
    y = step(z)
```

```
    return y
```

```
print(f"\n #### 1. STEP 함수 ####")
```

```
for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
```

```
    y_and = perceptron(x1, x2, 0.5, 0.5, -0.7)
```

```
    y_or = perceptron(x1, x2, 1, 1, -0.5)
```

```
    y_nand = perceptron(x1, x2, -1, -1, 1.5)
```

```
    y_xor = perceptron(y_or, y_nand, 0.5, 0.5, -0.7)
```

```
    print(f"입력:{x1},{x2} | AND:{y_and} | OR:{y_or} | NAND:{y_nand} | XOR:{y_xor}")
```

```
#2. sigmoid
```

```
def sigmoid_logicgate():
```

```
    def sigmoid(z):
```

```
        y = 1/(1+np.exp(-z))
```

```
        return y
```

```
def perceptron(A, B, w1, w2, b):
```

```
    z = w1*A + w2*B +b
```

```
    y = sigmoid(z)
```

```
    return y
```

```
def decision(v):
```

```
    if v>=0.5:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
print(f"\n #### 2. SIGMOID 함수 ####")
```

```

for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
    y_and = perceptron(x1, x2, 10, 10, -15)
    y_or = perceptron(x1, x2, 10, 10, -5)
    y_nand = perceptron(x1, x2, -10, -10, 15)
    y_xor = perceptron(y_or, y_nand, 10, 10, -15)

    print(f"입력:({x1},{x2}) | AND:[{decision(y_and)}, {y_and:.4f}] | OR:[{decision(y_or)}, {y_or:.4f}] | NAND:[{decision(y_nand)}, {y_nand:.4f}] | XOR:[{decision(y_xor)}, {y_xor:.4f}]")

```

#3. tanh

```
def tanh_logicgate():
```

```
    def tanh(z):
```

```
        y = np.tanh(z)
```

```
        return y
```

```
def perceptron(A, B, w1, w2, b):
```

```
    z = w1*A + w2*B + b
```

```
    y = tanh(z)
```

```
    return y
```

```
def decision(v):
```

```
    if v >= 0:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
print(f"\n#### 3. TANH 함수 ####")
```

```
for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
```

```
    y_and = perceptron(x1, x2, 10, 10, -15)
```

```
    y_or = perceptron(x1, x2, 10, 10, -5)
```

```
    y_nand = perceptron(x1, x2, -10, -10, 15)
```

```
    y_xor = perceptron(y_or, y_nand, 10, 10, -15)
```

```

    print(f"입력:({x1},{x2}) | AND:[{decision(y_and)}, {y_and:.4f}] | OR:[{decision(y_or)}, {y_or:.4f}] | NAND:[{decision(y_nand)}, {y_nand:.4f}] | XOR:[{decision(y_xor)}, {y_xor:.4f}]")

```

#4.ReLU

```
def relu_logicgate():
```

```
    def relu(z):
```

```
        if z > 0:
```

```
            return z
```

```
        else:
```

```
            return 0.0
```

```
def perceptron(A, B, w1, w2, b):
    z = w1*A + w2*B + b
    y = relu(z)
    return y
```

```
def decision(v):
    if v > 0:
        return 1
    else:
        return 0
```

```
print(f"\n#### 4. RELU 함수 결과 ####")
for x1, x2 in [(0, 0), (1, 0), (0, 1), (1, 1)]:
    y_and = perceptron(x1, x2, 1, 1, -0.7)
    y_or = perceptron(x1, x2, 1, 1, -0.5)
    y_nand = perceptron(x1, x2, -1, -1, 1.5)
    y_xor = perceptron(y_or, y_nand, 1, 1, -0.7)
```

```
print(f"입력:({x1},{x2}) | AND:[{decision(y_and)}, {y_and:.2f}] | OR:[{decision(y_or)}, {y_or:.2f}] | NAND:[{decision(y_nand)}, {y_nand:.2f}] | XOR:[{decision(y_xor)}, {y_xor:.2f}]")
```

```
step_logicgate()
sigmoid_logicgate()
tanh_logicgate()
relu_logicgate()
```